DSC 40B Discussion 4 Worksheet

Friday, April 24, 2026

Imagine you are a tutor, and you need to explain something to someone who has only a vague understanding of what's going on. Do your best to get the point across. **ALTERNATE**.

If you're not sure, ask your peer if they know how to explain it, then REPEAT the explanation back. Remember, the point of the exercise is to **vocalize** your thoughts.

If you are both unsure, make sure to come to/tutor for help! Do not use ChatGPT or any other LLM! The point of discussion is to prepare you for the exams.

Have one person in the pair open the Discussion 1 Gradescope assignment. After both you **and** your partner finish a problem **and explain it**, answer the corresponding multiple choice/short answer questions for both versions.. You will receive instant feedback on the questions, whether you are correct or not. If your answer is incorrect, please go back to the question to understand it and feel free to ask tutors about anything you are confused about. Once you have finished the entire worksheet or discussion is ending, turn in your assignment and add your partner to the submission.

Quick Icebreaker

Introduce yourself to your partner by sharing name, year, major, etc. How did you spend your weekend? (Do not spend more than 2 minutes!)

Questions

Question 1:

First Person:

Which of the following is NOT a valid loop invariant for insertion sort on arr? Explain your reasoning.

A. arr[0, …, i−1] is sorted
B. arr[0, …, i−1] contains the same elements as originally
C. arr[i, …, n−1] is unsorted
D. All elements left of i are ≤ all elements right of i

Second Person:

Which of the following is NOT a valid loop invariant for selection sort on arr at the start of iteration i? Explaining your reasoning.

A. The subarray arr[0, …, i−1] is sorted

B. The subarray arr[0, …, i−1] contains the i-th smallest elements in the array
C. The minimum element of arr[i, …, n−1] will be placed at position i by the end of this iteration
D. The subarray arr[i, .., .n−1] may be in arbitrary order

Question 2:

Second Person:

State and solve the recurrence relation describing the function's run time. Explain your reasoning and show your work.

```
def foo1(n):
    if n <= 1:
        return 1

    for i in range(n):
        for j in range(n):
            print('foo')

    return foo1(n - 2)
```

First Person:

State and solve the recurrence relation describing the function's run time. Explain your reasoning and show your work.

```
def foo2(n):
    if n <= 0:
        return 1

    x = foo2(n - 1)
    y = foo2(n - 1)
    return x + y
```

Question 3:

First Person:

Suppose that we start with an arbitrary list, then the worst case asymptotic runtime of merge sort is faster than that of insertion sort. Explain your reasoning.

- ☐ True
- ☐ False

Second Person:

Suppose that we start with an already sorted list, then the asymptotic runtime of merge sort is faster than that of insertion sort. Explain your reasoning.

- ☐ True
- ☐ False

Question 4:

Second Person:

Suppose that we start with an arbitrary list and perform a modified merge sort, where we split the list into three parts of roughly equal length, recursively sort each part, and then merge the 3 parts. Which of the following describes a recurrence relation for the runtime of this algorithm? Explain your reasoning and show your work.

- ☐ $T(n) = 3T(n/3) + n$
- ☐ $T(n) = T(n/3) + n$
- ☐ $T(n) = 3T(n) + n$
- ☐ $T(n) = 3T(n/3) + n^3$
- ☐ $T(n) = T(n/3) + n^3$
- ☐ $T(n) = 3T(n) + n^3$

First Person:

Solve the recurrence relation above. What is the asymptotic time complexity of this modified merge sort? Discuss how it compares to the asymptotic time complexity of standard merge sort. Explain your reasoning and show your work.

- ☐ $T(n) = \Theta(n)$
- ☐ $T(n) = \Theta(n^2)$
- ☐ $T(n) = \Theta(n^3)$
- ☐ $T(n) = \Theta(nlog(n))$
- ☐ $T(n) = \Theta(n^2log(n))$
- ☐ $T(n) = \Theta(n^3log(n))$

Question 5:

Example tracing merge sort on the following list: [4, 2, 1, 3]
[4, 2, 1, 3]
[4, 2] [1, 3]
[4] [2] [1] [3]

[2, 4] [1, 3]
[1, 2, 3, 4]

First Person:

Referring to the example above and implementation in lecture, trace out running merge sort on the following list: [4, 9, 1, 25, 8, 7, 15, 5].

How many calls of `mergesort()` and `merge()`?

Second Person:

Referring to the example above and implementation in lecture, trace out running merge sort on the following list: [10, 4, 17, 2, 9, 1, 13, 6, 8].

How many calls of `mergesort()` and `merge()`?

Question 6:

Second Person:

Suppose we are computing the k-th order statistic of a list of numbers by pivoting on the **middle** element of the list. For which of the following is quickselect expected to perform the **worst**? Explain your reasoning.

- ☐ Sorted list
- ☐ Partially sorted list
- ☐ List sorted in reverse order
- ☐ Arbitrary list

First Person:

Suppose we are computing the k-th order statistic of a list of numbers by pivoting on the **first** element of the list. For which of the following is quickselect expected to perform the **best**?

- ☐ Sorted list
- ☐ Partially sorted list
- ☐ List sorted in reverse order
- ☐ Arbitrary list

Question 7:

Example tracing of `quickselect()` using Approach #1 in lecture to find the 3rd largest element in [3, 1, 4, 2]:

qss(3, 0, 4)
    [3, 1, 4, **2**] → [1, **2**, 4, 3]
qss(3, 2, 4)
    [1, 2, 4, **3**] → [1, 2, **3**, 4]
→ Return 3 (as pivot is in the target position)

First Person:

Refer to the implementation of `quickselect()` in lecture. Instead of selecting a random index as the pivot index, select the largest index possible. Trace the calls of `quickselect()` to find the 5th largest element in the [4, 9, 1, 25, 8, 7, 5]. Use Approach #1 in lecture.

Second Person:

Refer to the implementation of `quickselect()` in lecture. Instead of selecting a random index as the pivot index, select the largest index possible. Trace the calls of `quickselect()` to find the 5th largest element in the [17, 2, 9, 1, 13, 6, 8]. Use Approach #1 in lecture.